DAM 2º • APUNTES DE ÉPOCA PRIME

APUNTES PROFESIONALES: DISEÑO DE INTERFACES (DI)

Material de Alto Rendimiento para Memorizar y Repasar de un Vistazo

1. PRINCIPIOS DE DISEÑO Y USABILIDAD (UX / UI)

a) **Diferenciación conceptual**:

- **UI (*User Interface*)**: Aspecto visual, estético y formal con el que interactúa el usuario (botones, tipografías, colores, disposición).
- **UX (*User Experience*)**: Percepción global, nivel de satisfacción, facilidad de aprendizaje y eficiencia del usuario al interactuar con el producto.

b) **Heurísticas esenciales de Jakob Nielsen**:

- **Visibilidad del estado del sistema**: Informar en todo momento de qué está ocurriendo mediante feedback claro y oportuno.
- **Correspondencia con el mundo real**: Utilizar palabras, frases y conceptos familiares al usuario en lugar de jerga interna.
- **Control y libertad del usuario**: Soporte inmediato para deshacer y rehacer acciones accidentales (salidas de emergencia).
- **Consistencia y estándares**: Seguir las mismas convenciones y patrones en toda la aplicación.
- **Prevención de errores**: Diseñar para evitar fallos antes de que ocurran en lugar de limitarse a mostrar mensajes de error.
- **Reconocimiento antes que recuerdo**: Hacer visibles los elementos y acciones para reducir la carga de memoria de trabajo.
- **Flexibilidad y eficiencia de uso**: Proporcionar atajos y aceleradores para usuarios avanzados.
- **Diseño estético y minimalista**: Eliminar información irrelevante o rara vez necesaria ("menos es más").
- **Reconocimiento y recuperación de errores**: Mensajes redactados en lenguaje claro que indiquen la causa y propongan una solución.
- **Ayuda y documentación**: Instrucciones concisas, fáciles de buscar y centradas en las tareas reales.

c) **Accesibilidad universal (WCAG)**:

- **Contraste de color**: Ratio mínimo de 4.5:1 para texto normal y 3:1 para texto grande (nivel AA).
- **Independencia del color**: La información crítica nunca debe transmitirse únicamente mediante el color.

2. COLOR, TIPOGRAFÍA Y SISTEMAS DE RETÍCULA (LAYOUT)

a) **Teoría del color y armonía**:

- **Monocromática**: Variaciones de saturación y brillo de un único tono.
- **Análoga**: Colores vecinos en la rueda cromática; sensación de calma y cohesión.
- **Complementaria**: Colores opuestos en la rueda; alto contraste y llamada de atención.
- **Regla 60-30-10**: 60% color dominante neutro (fondo), 30% color secundario (estructuras), 10% color de acento (CTAs e interacciones clave).

b) **Tipografía**:

- **Familias tipográficas**:

  - *Serif*: Tipografías con remates; asociadas a tradición y lectura impresa prolongada.

  - *Sans-Serif*: Tipografías sin remates (ej. Roboto, Inter); estándar en pantallas por legibilidad en bajas resoluciones.

  - *Monospace*: Ancho de caracteres fijo; código fuente y datos tabulares.

- **Escala modular y jerarquía**: Proporciones matemáticas consistentes entre titulares (H1, H2, H3), subtítulos y texto corrido.

c) **Retículas y espaciado (Layout)**:

- **Sistema de cuadrícula base (Grid de 8pt / 4pt)**: Todos los márgenes, rellenos (*padding*) y dimensiones deben ser múltiplos de 8 o 4 para garantizar consistencia entre densidades de pantalla.
- **Espacio en blanco (*Negative Space*)**: Elemento activo de diseño que previene la sobrecarga cognitiva y agrupa elementos relacionados (Ley de Proximidad de Gestalt).

3. ARQUITECTURA DE LA INFORMACIÓN, WIREFRAMES Y FIGMA

a) **Fases de prototipado**:

- **Bocetos (*Sketches*)**: Dibujos rápidos en papel de baja fidelidad para explorar ideas sin coste.
- **Wireframes**: Esquemas estructurales en escala de grises de media fidelidad centrados en la disposición, jerarquía y navegación sin distracciones visuales.
- **Prototipos interactivos**: Modelos de alta fidelidad con microinteracciones, tipografía y paleta final testeables con usuarios reales.

b) **Herramientas de diseño (Figma)**:

- **Componentes**: Elementos reutilizables en todo el proyecto; un cambio en el componente maestro (*Master*) se propaga a todas sus instancias.
- **Variantes**: Agrupación de diferentes estados de un componente (ej. botón activo, presionado, deshabilitado).
- **Auto Layout**: Sistema responsivo que adapta automáticamente el contenedor al tamaño de su contenido interior simulando el modelo de cajas del desarrollo real.

4. FUNDAMENTOS DEL LENGUAJE DART

a) **Características nucleares**:

- Lenguaje orientado a objetos, fuertemente tipado, con compilación anticipada (AOT) y en tiempo de ejecución (JIT).

b) **Tipos de datos y variables**:

- Tipos básicos: `int`, `double`, `String`, `bool`, `List`, `Map`, `Set`.
- Inferencia: `var` infiere el tipo en la asignación inicial; `dynamic` desactiva la comprobación estática.
- Inmutabilidad: `final` se evalúa una sola vez en tiempo de ejecución; `const` es constante estricta evaluada en tiempo de compilación.

c) **Null Safety**:

- Por defecto, las variables no admiten nulos (`String texto = "hola";`).

- Operador `?` : Declara variable nulable (`String? texto = null;`).
- Operador `!` : Aserción de no nulo; fuerza al compilador a tratar una expresión como no nula.
- Operador `??` : Asigna un valor por defecto si la expresión es nula (`val = dato ?? "defecto";`).

d) **Programación asíncrona**:

- `Future` : Representa un valor que estará disponible en un instante posterior.
- `async` / `await` : Sintaxis moderna para gestionar asincronía sin bloquear el hilo principal.

5. ARQUITECTURA Y FUNDAMENTOS DE FLUTTER

a) **Principio fundacional**: "En Flutter, todo es un Widget".

b) **Árboles de Flutter**:

- **Widget Tree**: Configuración inmutable de la interfaz descrita en código declarativo.
- **Element Tree**: Gestiona el ciclo de vida y coordina los widgets con su renderizado.
- **RenderObject Tree**: Dibuja físicamente píxeles en pantalla controlando tamaño y geometría.

c) **Comandos de terminal imprescindibles**:

- `flutter doctor` : Diagnostica dependencias, SDKs y herramientas instaladas.
- `flutter create` : Genera la estructura de un proyecto Flutter nuevo.
- `flutter run` : Compila y lanza la aplicación en el dispositivo o emulador seleccionado.
- `flutter pub get` : Descarga y resuelve las dependencias listadas en `pubspec.yaml` .

d) **Archivo** `pubspec.yaml` : Fichero de configuración del proyecto donde se declaran metadatos, versiones del SDK, dependencias de paquetes y recursos externos (imágenes, fuentes).

6. CATÁLOGO DE WIDGETS Y CICLO DE VIDA

a) **Clasificación por estado**:

- `StatelessWidget` :

  - Widget inmutable cuya configuración visual no cambia dinámicamente en tiempo de ejecución.

  - Método central: `Widget build(BuildContext context)` .

- `StatefulWidget` :

  - Widget mutable asociado a un objeto `State` que almacena datos cambiantes.

  - Ciclo de vida: `createState()` -> `initState()` -> `build()` -> `dispose()` .

  - `setState(() { ... })` : Notifica al framework que el estado interno ha mutado, forzando la reconstrucción del subárbol visual.

b) **Widgets estructurales y de disposición**:

- `Scaffold` : Proporciona la estructura visual básica de Material Design (`appBar` , `body` , `floatingActionButton` , `drawer`).
- `AppBar` : Barra superior para títulos, navegación y acciones.
- `Column` / `Row` : Alineación vertical u horizontal de una lista de widgets hijos.
- `Container` : Caja multipropósito con soporte para márgenes, rellenos, decoraciones y bordes.
- `SizedBox` : Espaciador rígido de dimensiones exactas.

- `Padding` : Aplica separación interior a cualquier widget hijo.
- `ListView` : Lista de desplazamiento vertical/horizontal; optimizado con `ListView.builder` para colecciones masivas.

c) **Navegación**:

- `Navigator.push(context, MaterialPageRoute(...))` : Inserta una nueva pantalla en la pila de navegación.
- `Navigator.pop(context)` : Extrae y elimina la pantalla superior actual volviendo a la anterior.

GLOSARIO DE TÉRMINOS CLAVE

UI (User Interface): Capa visual interactiva que media la comunicación entre humano y máquina.

UX (User Experience): Conjunto de factores relativos a la interacción, percepción y facilidad de uso del producto.

Heurística de usabilidad: Principio empírico de diseño validado para crear interfaces intuitivas y eficientes.

Wireframe: Representación visual esquemática en escala de grises de la estructura de una interfaz.

Auto Layout: Motor de alineación y espaciado adaptativo de contenedores visuales en Figma.

Null Safety: Mecanismo de seguridad en compilación que previene fallos por punteros nulos.

Widget: Unidad atómica inmutable de construcción de interfaz en el framework Flutter.

StatelessWidget: Widget estático cuya apariencia depende exclusivamente de los parámetros de entrada.

StatefulWidget: Widget dinámico capaz de redibujarse cuando cambian sus datos internos mediante `setState()`.

Scaffold: Estructura de layout predefinida que implementa el esqueleto visual de Material Design.

ESQUEMA RESUMEN CONCEPTUAL

1. PRINCIPIOS UX/UI

- UI (visual) vs UX (experiencia)
- 10 Heurísticas de Nielsen (visibilidad, consistencia, prevención de errores)
- Accesibilidad WCAG (contraste 4.5:1)

2. COLOR, TIPOGRAFÍA Y RETÍCULA

- Regla 60–30–10 y combinaciones cromáticas
- Familias tipográficas (Sans-Serif en pantalla)
- Sistema de espaciado modular de 8pt

3. PROTOTIPADO Y FIGMA

- Bocetos -> Wireframes -> Prototipos interactivos
- Componentes, Variantes y Auto Layout

4. LENGUAJE DART

- Tipado estático y tipos inmutables (`final` / `const`)
- Null Safety (`?` , `!` , `??`)
- Programación asíncrona (`Future` , `async` , `await`)

5. ARQUITECTURA FLUTTER

- "Todo es un Widget" (Widget Tree, Element Tree, RenderObject Tree)
- Herramientas de consola (`flutter run` , `flutter doctor`)
- Gestión de dependencias en `pubspec.yaml`

6. WIDGETS Y ESTADO

- `StatelessWidget` (estático) vs `StatefulWidget` (dinámico con `setState`)
- Estructura: `Scaffold` , `AppBar` , `Column` , `Row` , `ListView`
- Navegación por pila: `push()` y `pop()`